

Guía Didáctica: Promesas en JavaScript

Apartado 2: Cómo se crea una promesa en JavaScript

Explicación

En JavaScript, se puede crear una **promesa personalizada** utilizando el constructor `Promise`. La sintaxis básica es:

```
const promesa = new Promise((resolve, reject) => {
  // Operación asíncrona
  if(todoBien){
    resolve(valor); // se cumple
  } else {
    reject(error); // se rechaza
  }
});
```

- **resolve(valor):** indica que la promesa se completó correctamente y devuelve un valor.
- **reject(error):** indica que la promesa falló y devuelve un error.
- Una vez creada, se maneja con `then()`, `catch()` y `finally()`.

Ejemplo práctico 1: Crear una promesa simple

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Crear Promesa</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">
  <h1>Ejemplo 1: Crear Promesa Simple</h1>
  <button id="btnCrear1" class="btn btn-primary">Ejecutar
Promesa</button>
  <div id="resultadoCrear1" class="mt-3"></div>

  <script>
    const btnCrear1 = document.getElementById('btnCrear1');
    const resultadoCrear1 =
document.getElementById('resultadoCrear1');

    btnCrear1.addEventListener('click', () => {
      const promesa = new Promise((resolve, reject) => {
        setTimeout(() => resolve("Promesa creada y
resuelta!"), 2000);
      });
```

```

        promesa.then(msg => {
            resultadoCrear1.innerHTML = `<div class="alert alert-
success">${msg}</div>`;
        });
    });
</script>
</body>
</html>

```

Explicación:

- Creamos una promesa que se resuelve después de 2 segundos.
- `then()` recibe el mensaje y lo muestra en un div.

Ejemplo práctico 2: Crear promesa con error

```

<h2>Ejemplo 2: Promesa con Rechazo</h2>
<button id="btnCrear2" class="btn btn-warning">Ejecutar
Promesa</button>
<div id="resultadoCrear2" class="mt-3"></div>

<script>
const btnCrear2 = document.getElementById('btnCrear2');
const resultadoCrear2 = document.getElementById('resultadoCrear2');

btnCrear2.addEventListener('click', () => {
    const promesa = new Promise((resolve, reject) => {
        setTimeout(() => {
            const exito = false;
            exito ? resolve("Promesa resuelta") : reject("Promesa
rechazada!");
        }, 2000);
    });

    promesa
        .then(msg => resultadoCrear2.innerHTML = `<div class="alert
alert-success">${msg}</div>`)
        .catch(err => resultadoCrear2.innerHTML = `<div class="alert
alert-danger">${err}</div>`);
    });
</script>

```

Explicación:

- `reject()` se usa para simular fallo.
- `catch()` maneja el error y muestra un mensaje en pantalla.

Ejemplo práctico 3: Promesa encadenada

```

<h2>Ejemplo 3: Promesa Encadenada</h2>

```

```

<button id="btnCrear3" class="btn btn-secondary">Ejecutar
Chaining</button>
<div id="resultadoCrear3" class="mt-3"></div>

<script>
const btnCrear3 = document.getElementById('btnCrear3');
const resultadoCrear3 = document.getElementById('resultadoCrear3');

btnCrear3.addEventListener('click', () => {
  new Promise((resolve, reject) => {
    setTimeout(() => resolve(5), 1000);
  })
  .then(num => num * 2)
  .then(num => num + 3)
  .then(result => {
    resultadoCrear3.innerHTML = `<div class="alert alert-
success">Resultado final: ${result}</div>`;
  })
  .catch(err => {
    resultadoCrear3.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
  });
});
</script>

```

Explicación:

- Cada `then()` recibe el valor del anterior y permite hacer operaciones secuenciales.
- Muy útil cuando cada paso depende del resultado anterior.

Ejemplo práctico 4: Uso de Fetch API con promesas

La **Fetch API** usa promesas de forma nativa para hacer **peticiones HTTP**:

```

<h2>Ejemplo 4: Fetch API</h2>
<button id="btnFetch" class="btn btn-success">Cargar Datos</button>
<div id="resultadoFetch" class="mt-3"></div>

<script>
const btnFetch = document.getElementById('btnFetch');
const resultadoFetch = document.getElementById('resultadoFetch');

btnFetch.addEventListener('click', () => {
  fetch('https://jsonplaceholder.typicode.com/users')
    .then(response => response.json())
    .then(data => {
      let html = '<ul class="list-group">';
      data.forEach(user => {
        html += `<li class="list-group-item">${user.name}
(${user.email})</li>`;
      });
      html += '</ul>';
      resultadoFetch.innerHTML = html;
    })
});

```

```
        .catch(err => resultadoFetch.innerHTML = `<div class="alert alert-danger">${err}</div>`);
    });
</script>
```

Explicación:

- `fetch()` devuelve una **promesa**.
 - `then()` procesa la respuesta.
 - `catch()` captura errores de la petición.
 - Permite hacer **peticiones al servidor de forma asíncrona**, muy útil para datos reales.
-

Ejemplo práctico 5: Fetch con POST y promesa

```
<h2>Ejemplo 5: Fetch POST</h2>
<button id="btnPost" class="btn btn-info">Enviar Datos</button>
<div id="resultadoPost" class="mt-3"></div>

<script>
const btnPost = document.getElementById('btnPost');
const resultadoPost = document.getElementById('resultadoPost');

btnPost.addEventListener('click', () => {
    fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title: 'Prueba', body: 'Contenido de ejemplo', userId: 1 })
    })
    .then(response => response.json())
    .then(data => {
        resultadoPost.innerHTML = `<div class="alert alert-success">ID creado: ${data.id}</div>`;
    })
    .catch(err => {
        resultadoPost.innerHTML = `<div class="alert alert-danger">${err}</div>`;
    });
});
</script>
```

Explicación:

- `fetch()` con método POST envía datos JSON al servidor.
 - La respuesta se procesa usando promesas (`then()`) y se muestra en pantalla.
-

Ejercicios prácticos resueltos para alumnos

1. Crear una promesa que se resuelva después de 4 segundos y muestre un mensaje en pantalla.

2. Usar `fetch()` para obtener todos los posts de `jsonplaceholder` y mostrarlos en una lista.
3. Crear una promesa que genere un número aleatorio y lo multiplique en un `then()` y sume en otro.
4. Usar `fetch()` POST para enviar datos y mostrar el ID del recurso creado.

Ejercicios propuestos para alumnos

1. Crear una promesa que se cumpla solo si un número aleatorio es mayor que 0.7, y mostrar un mensaje distinto según éxito o fallo.
2. Usar `fetch()` GET para obtener usuarios y filtrarlos por email que contenga ".org".
3. Encadenar tres promesas que modifiquen un número (multiplicar, restar, sumar) y mostrar resultado final.
4. Usar `fetch()` POST para enviar un comentario simulado y mostrar en pantalla todos los datos recibidos.